



BrokerVerse Schedules and Batch Jobs

Scheduled jobs, batch processes

Prepared by – **iorta TechNXT**

Version - **1.0**

Date - **03 October 2026**

TABLE OF CONTENTS

1. Document Control.....	4
2. Acronyms.....	4
3. About this document.....	6
3.1 Terms used.....	6
4. How the scheduler works.....	7
4.1 Engine.....	7
4.2 Master > Schedules.....	8
5. Scheduled jobs.....	9
5.1 Summary.....	9
5.2 E-mail outbox.....	10
5.3 Renewal notice queue.....	10
5.4 Policy expiry.....	10
5.5 Quotation expiry.....	11
5.6 Renewal pipeline.....	11
5.7 Renewal notices.....	11
5.8 Receivable ageing.....	11
5.9 Collection reminders.....	12
5.10 Daily reports.....	12
5.11 Scheduled reports.....	12
5.12 Housekeeping.....	12
5.13 Dormant accounts.....	13
5.14 Month-end close reminder (off).....	13
5.15 Recurring journals (off).....	13
5.16 Accrual auto-reversal (off).....	13
5.17 Period auto soft-close (off).....	14
5.18 Bank reconciliation auto-match (off).....	14
6. Monitoring and failure handling.....	15
6.1 What to watch.....	15
6.2 When a job fails.....	15
6.3 Restart safety at a glance.....	15
7. Batch processes.....	17
7.1 Upload templates and the import rules.....	17
7.2 Renewal batch.....	18
7.3 Bulk disbursement and bulk print.....	19
7.4 Remittance processing.....	19
7.5 Month-end close run.....	20

7.6 Recurring journals.....	21
7.7 Bank statement import and auto-match.....	21
7.8 Insurer statement import.....	21
8. Operational run book.....	23
8.1 Overnight and early morning (automatic).....	23
8.2 Daily checklist.....	23
8.3 Month-end sequence.....	23
9. Gaps and observations.....	25
10. Document Sign-Off.....	26

1. Document Control

Date	Version	Author & Business Unit Info	Reviewed By & Business Unit Info	Approved By & Business Unit Info	Change History
03 October 2026	1.0	iorta TechNXT			Initial issue

2. Acronyms

Acronym / Symbol	Definition / Meaning
OOTB	Out of the box
BIR	Bureau of Internal Revenue
GL	General ledger
JV	Journal voucher
OR	Official receipt
PV	Payment voucher
SMTP	Simple Mail Transfer Protocol
CSV	Comma-separated values
XLSX	Excel workbook
PDF	Portable document format
API	Application programming interface
EWT	Expanded withholding tax
FX	Foreign exchange

3. About this document

This document describes everything BrokerVerse OOTB runs on a timetable and every batch process an operator starts by hand: what each one reads and writes, the setting that controls it, the e-mails and notifications it produces, how to run it on demand, how to watch it and what happens when it fails or runs twice. It closes with the daily and month-end run book.

It is written for the operations and finance supervisors who own the processes and for the support team that runs the platform. All facts come from the application code (backend/src/jobs, backend/src/modules/*/jobs.js, the module services), the seeded scheduled_jobs table and the upload templates in docs/templates.

3.1 Terms used

Term	Meaning
Scheduled job	A row of scheduled_jobs: code, name, cron expression, handler, parameters, enabled flag. Shown on Master > Schedules.
Job run	One execution of a scheduled job, recorded in job_runs (started, finished, status, output or error, triggered by).
Background queue	The table job_queue, used for long batches started from a screen (renewal batch notices and quotes).
Batch process	A process an operator starts on a screen that handles many records at once (uploads, bulk print, renewal batch, month-end close).
Business date	Today's date in the time zone of the setting general.timezone (Asia/Manila). Every job uses it, never the server date.

4. How the scheduler works

4.1 Engine

The scheduler starts with the API server (`backend/src/server.js`). It loads every enabled row of `scheduled_jobs` and schedules it with `node-cron` in the business time zone, so `0 6 * * *` runs at 06:00 Manila time whatever the server clock (containers run on UTC).

Item	Behaviour	Controlled by
Time zone	All cron expressions are read in <code>general.timezone</code> (Asia/Manila in the seed). Changing the setting reschedules every job.	Master > Configuration > Company & Branding > General
Scheduler on or off per server	Each API instance runs the scheduler unless the environment variable <code>SCHEDULER_ENABLED</code> is <code>false</code> , <code>0</code> , <code>no</code> or <code>off</code> . Use it to keep a web-only instance from running jobs. Run now still works on such an instance.	Environment variable
Picking up changes	Every instance checks every <code>SCHEDULER_RELOAD_SECONDS</code> (default 30) whether any job's cron, enabled flag, handler or parameters, or the time zone, changed, and reloads its timetable.	Environment variable
One run at a time	Each run takes a PostgreSQL advisory lock on the job code. A second instance that fires the same job at the same time skips it. A scheduled run also skips when another instance already started a scheduled run of the same job in the same minute. A skipped run records nothing.	Automatic
Missed runs	A run that falls while no instance is running is not caught up. The next scheduled time runs normally. Jobs that work on "everything due up to today" (policy expiry, recurring journals, accrual reversal, ageing) recover on their next run; jobs that look at one exact day (renewal notices) do not.	Design
Run history	Each run writes a <code>job_runs</code> row and updates Last run and Last status on the job. Run history is kept for housekeeping. <code>job_runs_days</code> (90 days).	Master > Configuration > Data Retention & Uploads

4.2 Master > Schedules

The screen lists every job with: Job, What it does, Schedule (in words, with the time zone in the column header), Status (Scheduled or Switched off), Next run, Last run, Last status, and three actions.

Action	What it does	Who may use it
Run now (play icon)	Runs the job at once on the server you are connected to, records a run with Triggered by = your user name, and shows the result as a message.	Accounting role or an administrator role
Run history (clock icon)	Shows the last 100 runs: Started, Finished, Status, Triggered by, Result (the output in words, or the error).	Permission read:schedules (Accounting, System Administrator)
Edit schedule (pencil icon)	Changes Schedule (cron) and Enabled. The dialog explains the cron format: minute hour day month weekday, for example 0 6 * * * runs daily at 06:00 Asia/Manila time. Save reloads the timetable at once.	Accounting role or an administrator role

Job parameters (for example the days before period end of the month-end reminder, or the list of reports of the daily-reports job) are not on the Edit dialog. A System Administrator changes them through the API: PUT /api/schedules/<code> with {"params": {...}}.

The Edit dialog does not check the cron expression. An invalid expression is saved, and the job is then left out of the timetable with only a warning in the server log. After every edit, check that Next run shows a date.

To run a job on demand:

1. Open Master > Schedules.
2. Find the job and click Run now.
3. Read the message: success with the job's output (for example "notifications: 3"), or the error.
4. Open Run history to see the run recorded with your user name.

A job switched off can still be run with Run now. This is the way to use the period-end and bank jobs that ship switched off without scheduling them.

5. Scheduled jobs

5.1 Summary

Job (code)	Default timing (Asia/Manila)	OOTB	Handler	Setting that controls it
E-mail outbox (email-outbox)	Every 5 minutes	On	emailOutbox	notification.email_enabled, notification.from_address, SMTP_URL
Renewal notice queue (renewal-queue)	Every minute	On	processRenewalQueue	none
Policy expiry (policy-expiry)	Daily 00:15	On	policyExpiry	none
Quotation expiry (quote-expiry)	Daily 00:30	On	quoteExpiry	limits.quote_validity_days (30)
Accrual auto-reversal (accrual-reversal)	00:30 on day 1 of each month	Off	accrualReversal	none
Recurring journals (recurring-journals)	Daily 01:15	Off	recurringJournals	the recurring journal templates
Dormant accounts (dormant-users)	Daily 01:45	On	dormantUsers	access.dormant_days (90)
Period auto soft-close (period-auto-soft-close)	Daily 02:00	Off	periodAutoSoftClose	job parameter graceDays (5)
Housekeeping (housekeeping)	Daily 02:45	On	housekeeping	housekeeping.*
Daily reports (daily-reports)	Daily 05:00	On	dailyReports	job parameter reports
Renewal pipeline (renewal-pipeline)	Daily 05:30	On	renewalPipeline	renewals.pipeline_days (90), renewals.grace_period_days (30)
Bank reconciliation auto-match (bank-auto-match)	Daily 05:45	Off	bankAutoMatch	bank_reconciliation.date_window_days, bank_reconciliation.group_max_lines, match rules
Renewal notices (renewal-notices)	Daily 06:00	On	renewalNotices	notification.renewal_reminder, limits.renewal_notice_days (60, 30, 15)
Receivable ageing (receivable-ageing)	Daily 07:00	On	receivableAgeing	limits.receivable_ageing_buckets (30, 60, 90, 120)
Month-end close reminder (month-end-reminder)	Daily 08:00	Off	monthEndReminder	job parameter daysBefore (3)
Collection reminders (collection-reminders)	Daily 08:00	On	collectionReminders	collections.reminder_days_before (7), collections.reminder_repeat_days (7), collections.email_subject, collections.email_template
Scheduled report (report- <code><id></code>)	As set on the report schedule	Created per schedule	scheduledReport	the report schedule; reports.email_subject, reports.email_body

The five finance jobs (accrual reversal, recurring journals, period auto soft-close, bank auto-match, month-end reminder) ship switched off. The finance team decides at go-live whether to switch them on or to run them from the screens.

5.2 E-mail outbox

Item	Detail
Purpose	Delivers the e-mails that the rest of the system has queued in <code>email_outbox</code> .
Reads	<code>email_outbox</code> rows with status <code>queued</code> and fewer than 5 attempts, oldest first, at most 50 per run.
Writes	Status sent with sent date, or the error and the attempt count; after the fifth failed attempt the status becomes <code>failed</code> .
Sends	Every queued e-mail, from the address in <code>notification.from_address</code> (OOTB BrokerVerse <connect@iortatechnxt.com>), over the SMTP server in <code>SMTP_URL</code> .
Switch	Nothing leaves the system unless <code>notification.email_enabled</code> is on and <code>SMTP_URL</code> is set. Otherwise the run reports how many are queued and why (" <code>notification.email_enabled</code> is false" or " <code>SMTP_URL</code> not set") and the e-mails wait.
On demand	Run now, or Retry on a message in Master > E-mail Outbox (resets its attempts and sends it at once when sending is enabled).
Restart safety	Each message is marked sent right after the SMTP server accepts it. A crash between the two steps can send that one message twice.

5.3 Renewal notice queue

Item	Detail
Purpose	Works through the background queue <code>job_queue</code> : renewal notices and renewal quotes started from Operations > Renewals > Renewal Batch.
Reads	Waiting jobs (up to 500 per run); jobs stuck in processing for more than 15 minutes are put back to waiting first.
Writes	Per batch policy: notice status <code>Sent</code> or <code>Failed</code> (with the error), quote status <code>Quoted</code> or <code>Failed</code> ; the renewal, its notice and activity records; the batch status (<code>Completed</code> , or back to <code>Draft</code>).
Sends	One renewal notice e-mail per policy (see the Communication Templates document) and an in-app notification to the policy owner.
Restart safety	A policy is only processed while its status is <code>Queued</code> , so a requeued job skips what was already sent. A policy being sent at the moment of a crash may receive its notice twice.

New queue jobs are also started at once when they are created; the every-minute job is the safety net after a restart.

5.4 Policy expiry

Sets the status of every active or issued policy whose expiry date is before the business date to `expired`. Output: number expired. Writes `policies.status`. Sends nothing. Safe to run any number of times.

5.5 Quotation expiry

Sets to expired every quotation in draft, quoted or sent status created more than `limits.quote_validity_days` (30) days ago. Output: number expired. Writes `quotes.status`. Sends nothing. Safe to rerun.

The job counts from the creation date of the quotation, not from its Valid Until date. A quotation whose validity was extended on screen still expires 30 days after it was created.

5.6 Renewal pipeline

Item	Detail
Purpose	Feeds the renewal queue and lapses renewals that were not completed in time.
Reads	Active and issued policies, not yet renewed, expiring between <code>renewals.grace_period_days</code> (30) days ago and <code>renewals.pipeline_days</code> (90) days ahead, with no renewal record; open renewals whose policy expired more than the grace period ago.
Writes	New renewal records (<code>renewals</code>), lapsed renewals with the reason "Not renewed within the 30-day grace period".
Notifications	For each lapse, an alert to the policy owner: "Policy lapsed: <policy number>".
Output	created, lapsed.
Restart safety	A policy with a renewal record is never enrolled again; a lapsed renewal is no longer open. Safe to rerun.

5.7 Renewal notices

Creates in-app reminders "Renewal due in N days" for the policy owner of every active or issued policy that expires exactly N days after the business date, for each N in `limits.renewal_notice_days` (60, 30, 15). The message reads "Policy <number> expires on <date>" and links to the policy. Output: notifications created.

- Switched off by the setting `notification.renewal_reminder` (the run then reports that reminders are switched off).
- It sends no e-mail to the client. Client renewal notices go out from the Renewal Queue or a Renewal Batch.
- Safe to rerun: a reminder already created for the same policy and window is not repeated. Because it looks at one exact day, a day on which the job did not run is not caught up.

5.8 Receivable ageing

Recomputes, for every open or partly paid bill, the days past the due date and the ageing bucket from `limits.receivable_ageing_buckets` (labels current, 1-30, 1-60, 1-90, 1-120 and >120: each label starts at 1, so "1-60" means 31 to 60 days), and stores them on the bill (`receivables.age_days`, `ageing_bucket`). Output: updated. Safe to rerun.

The job description on the screen says "and notify collections", but the job sends no notification. Collection notifications come from the Collection reminders job.

5.9 Collection reminders

Item	Detail
Purpose	Reminds clients of premiums falling due or overdue and tells the account officer.
Reads	Collection items not Paid or Committed, due within <code>collections.reminder_days_before</code> (7) days or already overdue, not reminded in the last <code>collections.reminder_repeat_days</code> (7) days.
Writes	<code>collection_items.last_reminder_at</code> ; a Reminder action on the collection item.
Sends	An e-mail to the client (template <code>collections.email_subject</code> / <code>collections.email_template</code>) when the client has an e-mail address; an in-app reminder to the policy owner and to holders of <code>write:collections: "Premium due <date> - <policy>"</code> or <code>"Premium overdue N day(s) - <policy>"</code> .
Output	candidates, emails, notifications, skipped (clients without e-mail).
On demand	Run now, or the Send Payment Reminders Now button on Accounts > Collections.
Restart safety	The repeat window stops a second reminder within 7 days.

5.10 Daily reports

Generates the Production Register, Collection Report and Claims Position (or the reports named in the job parameter reports) with the default range (365 days back to today) and the default format (xlsx), and records them in the generated-file history. It also deletes generated files older than `reports.retention_days`. It does not e-mail the files. A rerun produces another set of files. See the Reports Book.

5.11 Scheduled reports

One job per report schedule, code `report-<schedule id>`, created when a report schedule is saved through the API. Each run generates the report and queues an e-mail with the download link to the schedule's recipients, then records last run and status on the schedule. A rerun e-mails again.

5.12 Housekeeping

Deletes, in batches of 5,000 rows, data past its retention period. Every period is a setting on Master > Configuration > Data Retention & Uploads (days; 0 keeps forever).

Data	Setting	OOTB days	Rows deleted
Job run history	<code>housekeeping.job_runs_days</code>	90	runs started before the cut-off
Sent e-mails	<code>housekeeping.email_outbox_sent_days</code>	180	sent messages
Failed e-mails	<code>housekeeping.email_outbox_failed_days</code>	730	failed messages

Data	Setting	OOTB days	Rows deleted
Sign-in history	housekeeping.login_history_days	365	sign-in records
Sign-in sessions	housekeeping.refresh_tokens_days	30	revoked or expired refresh tokens
Password reset codes	housekeeping.password_resets_days	7	used or expired codes
Read notifications	housekeeping.notifications_read_days	180	read notifications only
Background queue	housekeeping.job_queue_done_days	30	completed queue jobs
Audit trail	housekeeping.audit_log_days	0 (keep)	never below 2,557 days (7 years, BIR and Insurance Commission record keeping)

Output: rows deleted per table and the tables kept forever. Safe to rerun.

5.13 Dormant accounts

Deactivates every active user (except the built-in administrator BrokerVerse) who has not signed in, or since creation, for `access.dormant_days` (90) days, and ends their sessions. Output: number and user names. A deactivated user is reactivated on Master > Generals > User Management > User. Setting the days to 0 disables the job's effect. Safe to rerun.

5.14 Month-end close reminder (off)

Notifies holders of `write:period-end` (high priority, link to Month-End Close) when the current accounting period ends within `daysBefore` (3) days: "Month-end close: 2026-10 ends in 2 day(s)", message "Accounting period 2026-10 ends on ... Post pending journals and prepare the month-end close run."; and for every ended period still open: "Month-end close: 2026-09 is still open". A notification is not repeated for the same period and title.

5.15 Recurring journals (off)

Posts every active recurring journal template (Accounts > Period End > Recurring Journals) whose next run date has arrived, one journal per occurrence, catching up missed occurrences. Templates with auto-post create posted journals, the others pending journals that need approval. Templates with auto-reverse are scheduled for reversal the next day. Output: journals posted and errors. Idempotent: one journal per template and occurrence date; a failed occurrence keeps the next run date there and is retried. The same action is on the screen: Accounts > Period End > Recurring Journals > Run due.

5.16 Accrual auto-reversal (off)

On day 1 of the month, posts the reversal of every accrual, commission deferral and FX revaluation journal whose reversal date has arrived (journals created by the month-end close run or by auto-reverse templates). Each reversal is done on its own; a failure (for example a closed period) is reported and retried on the next run. Idempotent: an entry already reversed is not reversed again.

5.17 Period auto soft-close (off)

Soft-closes every open regular period that ended more than graceDays (5) days ago, when none of the automatic blocking month-end checks fails (no unposted or pending journals, trial balance balances, suspense account cleared). Manual sign-offs are not considered. Periods with failing checks are skipped and listed with the reasons. In a soft-closed period only users with approve:period-end may post.

5.18 Bank reconciliation auto-match (off)

Runs the automatic matching rules (adjustment, contra, reference, amount and date, one-to-many, many-to-one) on every active bank account linked to a GL cash account that has unmatched statement lines. Each account is processed on its own; an error on one account does not stop the others. It never matches into a period whose reconciliation is approved. Output: accounts, matches per account. Safe to rerun: it only looks at unmatched lines. Matching also runs right after each statement import when bank_reconciliation.auto_match_on_import is on (default), which is why the job ships switched off.

6. Monitoring and failure handling

6.1 What to watch

Where	What it shows
Master > Schedules	Last status per job (success or failed) and Next run. A blank Next run on an enabled job means the scheduler is off on that server or the cron is invalid.
Master > Schedules > Run history	Each run with its output or error message.
Master > E-mail Outbox	Queued, sent and failed e-mails, with the last error and Retry. A banner tells whether sending is enabled.
Operations > Renewals > Renewal Batch	Per batch: queued, sent, failed counts; Retry Failed.
Server log	"scheduled job <code> failed" with the run id and error; "scheduler: N job(s) scheduled (time zone Asia/Manila)" at start and after each reload.
Database	job_runs (status, error), job_queue (status, error, attempts), email_outbox (status, attempts, error).

6.2 When a job fails

A failed run is recorded with status failed and the error text, and the job's Last status turns to failed. Nothing is retried automatically: the job runs again at its next scheduled time.

1. Open Run history and read the error.
2. Fix the cause (for example a closed period for a recurring journal, a missing GL account, SMTP unreachable).
3. Click Run now. Every job in this document can be rerun; the restart-safety notes above say what a rerun repeats.

No e-mail or notification is sent when a scheduled job fails. Someone has to look at Master > Schedules each morning (see the run book), or the support team adds an alert on the server log line "scheduled job ... failed".

6.3 Restart safety at a glance

Job	Safe to rerun	What a rerun repeats
Policy expiry, quotation expiry, receivable ageing, housekeeping, dormant accounts, period auto soft-close	Yes	Nothing
Renewal pipeline, month-end reminder, renewal notices	Yes	Nothing (existing records and notifications are detected)
Recurring journals, accrual reversal, bank auto-match	Yes	Nothing (one journal per occurrence; reversed entries skipped; matched lines skipped)
Collection reminders	Yes	Nothing within the repeat window
E-mail outbox, renewal notice queue	Yes	At most the one message being sent at the moment of a

Job	Safe to rerun	What a rerun repeats
		crash
Daily reports, scheduled reports	Yes	Another file; a scheduled report e-mails its recipients again

7. Batch processes

7.1 Upload templates and the import rules

Every bulk upload has a template in docs/templates (also downloadable on the screens that have an Upload or Import button). Each workbook has three sheets: Data (header row and one or two Philippine sample rows, required columns in dark red), Columns (required or not, format, allowed values, example, other accepted header names) and Instructions (screen, API route, file types, row limit, what happens on errors).

Rules common to all uploads:

- XLSX (first sheet) or CSV saved as UTF-8; the first row holds the column headers. Column order does not matter; headers are matched without regard to case, spaces or punctuation.
- Dates as YYYY-MM-DD, amounts as plain numbers (no peso sign, no thousands separator). Delete the sample rows before uploading.
- File size up to 10 MB. Rows: up to 20,000 for the data-migration uploads (IMPORT_MAX_ROWS), up to 1,000 for receipts, payment vouchers, open items and masters (limits.bulk_upload_max_rows).
- Row by row (most uploads): each row is checked and saved on its own; the result lists the failed rows with their spreadsheet row number and the reason. Upload only the corrected rows again, otherwise the saved rows are created a second time.
- All or nothing (opening balances, bank statements): the whole file is checked first and nothing is saved when any row is wrong.

Upload	Screen	Rule	Notes
Leads	Operations > Leads/Prospects > Bulk Upload	Row by row	First Name or Company Name required.
Quotations	Operations > Quotation > Bulk Upload	Row by row	With a Lead Id the quotation joins that lead, else a new lead is created; premiums are recalculated from the rate and tariff.
Policies	Operations > Policy > Bulk Upload	Row by row	New business: client, policy (insurer at 100%), premium bill, booking journal and commission. Go-live mode ("Existing policies (go-live)"): client and policy only. Co-insured policies cannot be uploaded.
Official receipts	Accounts > Receipts > Bulk upload	Row by row	Each row issues an OR, applies it to the oldest open bills of the policy and posts the journal; the receipt date must be in an open period.
Payment vouchers	Accounts >	Row by row	Each row creates a

Upload	Screen	Rule	Notes
	Disbursement > Bulk upload		draft voucher, then approved and paid on screen (maker-checker).
Opening balances	Accounts > Period End > Period Management > Import opening balances	All or nothing	Debits must equal credits; loading again with the same go-live date replaces the earlier load; no journal is posted.
Open items	Accounts > Collections > Import open items	Row by row, rows already loaded skipped	Each row becomes an open bill of a go-live policy; no journal (the GL carries them in the opening balance).
Masters (29 templates)	The master screen > Upload, or the API route where the screen has no Upload button yet	Row by row	A row whose code exists is refused; edit existing records on screen.
Bank statement	Accounts > Bank Reconciliation > Reconciliation Workspace > Import statement	All or nothing	See bank statement import below.
Remittance bulk	Accounts > Remittance > Bulk Processing	Validate, then process valid rows	See remittance bulk processing below.
Remittance bank transactions	Accounts > Remittance > Reconciliation > Import	Read in the browser	TransDate, Reference, Amount, Description.
Users	Server script backend/scripts/provision-users.js	Dry run, then CONFIRM_PROVISION=yes	Not a screen upload; users must change the initial password.

The full column lists are in the templates and in docs/templates/README.md.

7.2 Renewal batch

Menu: Operations > Renewals > Renewal Batch. **Permission:** renewals write.

Step	What happens	Validation
Create batch	Pick policies by criteria (expiry range, insurer, product, premium range, client, payment status) or by list. A batch number is assigned and the batch is Draft.	At most <code>renewals.batch_max_policies</code> (500) policies; policies already renewed or cancelled, no longer renewable, or with a renewal in progress are refused or left out. Policies expiring up to <code>renewals.batch_window_days</code> (30) ahead are proposed.
Select	Tick the policies to act on.	
Send Renewal Notices	Queues the next notice (first, second, final) for the selected policies; the batch becomes Processing and the renewal notice queue sends them.	Only policies whose notice status is NotSent; notices must go in order when <code>renewals.enforce_notice_order</code> is on; the client needs an e-mail address.
Prepare quotes	Queues re-rated renewal	Only policies not yet quoted

Step	What happens	Validation
	quotes for the selected policies.	(or failed).
Retry Failed	Puts failed notices back in the queue.	Only Failed notices.
Generate Report	Downloads the batch with notice and quote status (Excel or CSV).	
Completion	When nothing is queued the batch becomes Completed if every selected policy had its notice, else back to Draft.	A batch with sent notices cannot be deleted, only cancelled.

7.3 Bulk disbursement and bulk print

- **Bulk Disburse** (Accounts > Disbursement > Bulk Disburse): creates one payout voucher, for approval, per selected agent or referrer with Approved commission lines. The vouchers then follow the normal approval and payment, and feed the QAP and Form 2307 (issued).
- **Bulk upload of vouchers**: see the upload table above.
- **Bulk print of payment vouchers** (Accounts > Disbursement > Bulk print): one PDF, one voucher per page, for one voucher or a customer code and date range.
- **Bulk print of official receipts** (Accounts > Receipts > Bulk print): one PDF, one receipt per page, for one receipt or a customer code and date range. The title comes from `receipts.print_title` ("Official Receipts").

7.4 Remittance processing

Process	Menu	Input	Output
Automated Processing	Accounts > Remittance > Automated Processing	Active automated remittance configurations (Remittance Master, for example ARM-001 Monthly Auto Remittance): insurers, frequency. Candidates are the unremitted policies per insurer.	Execute creates draft remittances; Process submits the selected drafts for approval as one batch and notifies holders of <code>write:remittance</code> ("Remittances awaiting approval").
Bulk Processing	Accounts > Remittance > Bulk Processing	A remittance file with the configuration BFM-001 (CSV) or BFM-002 (XLSX): PolicyNo, Premium, Commission, InsuredName.	Upload / Validate lists every row error (policy not found, premium not positive, commission above premium, duplicate policy); Process creates draft remittances from the valid rows.
Scheduling	Accounts > Remittance > Scheduling	Remittance schedules (Remittance Master), linked to automated configurations.	Run Now executes the linked automated remittance. Pause and resume.
Approval	Accounts > Remittance > Approval Workflow	Submitted remittances	Approve or reject (the approver must not be the submitter); the

Process	Menu	Input	Output
Statements	Accounts > Remittance > Statements	Period and insurers	initiator is notified. CSV statement, optionally e-mailed as a link.

The remittance schedules on Accounts > Remittance > Scheduling are not connected to the scheduler: they do not run on their own and do not appear on Master > Schedules. Their Next Run date is information only (the seeded schedules even carry the time zone EST). Run them with Run Now, or execute Automated Processing on the agreed day.

7.5 Month-end close run

Menu: Accounts > Period End > Month-End Close. **Permissions:** write:period-end to prepare; approve:period-end (Accounting Manager) to approve.

1. **Start** the close of a period. One open run per period; the checklist is copied into the run (MEC number).
2. **Execute** the steps: accruals, recurring journals, unearned commission deferral, FX revaluation, checks. Executing again first reverses the run's own earlier accrual, deferral and FX journals, so a rerun gives the same ledger as one run.
3. **Review the checklist.** Automatic checks: no unposted or pending journals (blocking), trial balance balances (blocking), suspense account cleared (blocking), no unapplied receipts, bank transactions reconciled, sub-ledgers tie out to the GL, issued policies are billed, remittances to insurers paid, direct-bill commission billed (warnings). Manual sign-offs: bank reconciliations reviewed and signed off (blocking), prepayments and depreciation reviewed, payroll and statutory contributions booked (warnings). Checklist items are maintained on Master > Finance > Close Checklist.
4. **Sign** the manual items.
5. **Submit** with the target soft-closed or closed. When `accounting.period_close_requires_approval` is on (default) the run waits for approval; otherwise the period closes at once.
6. **Approve** (a different user with approve:period-end) or **Reject** with a reason. Approval closes the period.
7. **Cancel** an unfinished run if needed; its accrual, deferral and FX journals are reversed.

The Month-End Close Status report shows every period with its run, failed checks, warnings, journals and who prepared and approved it.

The year-end close (Accounts > Period End > Year-End Close) needs all twelve periods closed and period 13 open. It posts the closing entries in period 13, writes the opening balances of the next year, locks the year and creates the next fiscal year.

7.6 Recurring journals

Templates are kept on Accounts > Period End > Recurring Journals (RJV number): name, kind (recurring or accrual), frequency (monthly, quarterly, yearly), start, next run and end dates, auto-post, auto-reverse, lines. They are posted by the Recurring journals job, by Run due on the same screen, or by the accruals and recurring steps of the month-end close run. See the job description above for idempotency.

7.7 Bank statement import and auto-match

Menu: Accounts > Bank Reconciliation > Reconciliation Workspace > Import statement. **Permission:** write:bank-reconciliation.

Item	Detail
Input	The bank's statement file (CSV or XLSX) with its format: BDO-SAMPLE, BPI-SAMPLE, MBT-SAMPLE for those banks' exports, GENERIC (template Bank_Statement_Generic_Upload_Template.xlsx) for others. Formats are kept on Master > Finance > Bank Statement Formats. Also: bank account, statement reference, opening and closing balance.
Preview	Shows the lines, the totals, the balance check and possible duplicates before saving.
Validation	Refused as a whole when a row cannot be read, when the statement does not balance (opening + credits - debits = closing) while bank_reconciliation.require_balanced_statement is on, or when the same file was already imported for the account. Duplicate lines can be skipped. Lines whose text starts with total, balance forward, beginning or ending balance are ignored.
Output	A bank statement (BST number) with its lines; then automatic matching when bank_reconciliation.auto_match_on_import is on. The message says how many lines were imported and auto-matched.
After	Match the rest by hand, book bank charges, interest and direct credits as adjustments, prepare the reconciliation (BRC number) and have it approved.

A statement can also be entered by hand (Manual statement), with the same checks.

7.8 Insurer statement import

Menu: Accounts > Insurer Reconciliation > Insurer Statements. **Permissions:** write:remittance to import and resolve; approve:insurer-reconciliation to approve.

Item	Detail
Input	The insurer's statement (CSV or XLSX): premium remittance confirmation or commission statement, with the insurer's format (Master > Finance > Insurer Statement Formats) or the generic one; insurer, type, period, the insurer's reference and the amount tolerance (insurer_reconciliation.amount_tolerance).
Preview	Lines, totals and rows that cannot be read;

Item	Detail
	nothing is saved.
Validation	A row that cannot be read stops the import; the same file cannot be imported twice for an insurer.
Auto-match	By policy number (ignoring case, spaces and dashes) to a remittance line or commission debit note line; otherwise on gross premium within the tolerance and insured name when exactly one record fits. Premium, commission, taxes and amount paid are compared within the tolerance.
Output	An insurer statement (ISR number) with matches and differences: lines not found at the broker, broker records missing on the statement, amount differences.
Resolution	Each difference is resolved with a note or an adjustment; submit (all resolved when <code>insurer_reconciliation.require_resolved</code>); a second user approves and the adjustments post through the posting rule <code>insurer_statement.adjustment</code> . The differences report downloads as Excel, CSV or PDF.

8. Operational run book

The run book lists what happens on its own and what the team checks or starts. Times are Asia/Manila.

8.1 Overnight and early morning (automatic)

Time	Job	Depends on
Every minute	Renewal notice queue	
Every 5 minutes	E-mail outbox	notification.email_enabled, SMTP
00:15	Policy expiry	
00:30	Quotation expiry	
00:30, day 1	Accrual auto-reversal (if switched on)	Month-end close run of the previous month
01:15	Recurring journals (if switched on)	Open period for the journal date
01:45	Dormant accounts	
02:00	Period auto soft-close (if switched on)	Blocking checks pass
02:45	Housekeeping	
05:00	Daily reports	
05:30	Renewal pipeline	Policy expiry has run
05:45	Bank auto-match (if switched on)	Statements imported
06:00	Renewal notices (in-app)	Renewal pipeline
07:00	Receivable ageing	
08:00	Collection reminders; month-end reminder (if switched on)	E-mail outbox for delivery

8.2 Daily checklist

1. **Master > Schedules:** every enabled job shows Last status success and a Next run. Open Run history for any failure and rerun after fixing.
2. **Master > E-mail Outbox:** no growing queue; resolve failed e-mails (wrong address, SMTP) and Retry.
3. **Operations > Renewals > Renewal Batch:** batches in Processing have moved on; Retry Failed where needed. Send the day's notices from the Renewal Queue.
4. **Accounts > Receipts and Collections:** post receipts; review overdue items after the 08:00 reminders.
5. **Accounts > Bank Reconciliation:** import yesterday's statements; review the auto-matches and match the rest.
6. **Accounts > Remittance:** run Automated Processing or Bulk Processing on the agreed days; approve submitted remittances.

8.3 Month-end sequence

Day	Step	Where
Last 3 days of the month	Month-end reminder (if on) prompts finance; post pending	Notifications; Accounts

Day	Step	Where
	journals, receipts and vouchers dated in the month.	
Last day	Import the bank statements up to the month end; match; book bank charges and interest.	Accounts > Bank Reconciliation
Day 1 to 3	Prepare and approve the bank reconciliation of each account for the month (the close needs the sign-off).	Accounts > Bank Reconciliation > Reconciliations
Day 1 to 3	Import and reconcile the insurer statements; approve adjustments.	Accounts > Insurer Reconciliation
Day 1 to 3	Run due recurring journals (if the job is off).	Accounts > Period End > Recurring Journals
Day 1 to 5	Raise commission debit notes for unbilled direct-bill commission; generate remittances due to insurers.	Accounts > Remittance
Day 3 to 5	Start the month-end close run, execute, review checks, sign, submit; manager approves.	Accounts > Period End > Month-End Close
After approval	Run Trial Balance (Opening / Movement / Closing), Income Statement, Balance Sheet, Month-End Close Status.	Reports > Financial Reports
Day 1 of next month	Accrual auto-reversal (job if on, otherwise it runs when a run is executed and the reversals are due).	Master > Schedules
Monthly / quarterly	VAT Summary, SLSP, QAP, SAWT and BIR Form 2307 for the BIR returns.	Accounts > Tax
Month end + grace (5 days)	Period auto soft-close (if on) soft-closes periods whose checks pass.	Master > Schedules

9. Gaps and observations

No.	Observation	Suggested action
1	No alert when a scheduled job fails; failures are visible only on Master > Schedules and in the server log.	Add an alert on the log line or a notification on failure; meanwhile check the screen daily.
2	The Edit dialog on Master > Schedules does not validate the cron expression; an invalid one silently drops the job from the timetable.	Validate on save; check Next run after each edit.
3	Job parameters (daysBefore, graceDays, reports) can only be changed through the API, though the period-end notes refer to Master > Schedules.	Add a parameters field to the Edit dialog.
4	Edit and Run now check the Accounting role, not the permission write:schedules that the System Administrator role holds.	Align the check with the permission.
5	The Renewal notices job only reaches policies expiring exactly 60, 30 or 15 days ahead; a missed day is not caught up. The job creates in-app reminders only.	Run the Renewal Queue daily; consider a window instead of an exact day.
6	The Receivable ageing job description says it notifies collections; it does not.	Correct the description or add the notification.
7	Quotation expiry counts from the creation date, not from Valid Until.	Decide which rule the broker wants.
8	Remittance schedules (Accounts > Remittance > Scheduling) are not run by the scheduler; seeded with time zone EST.	Run with Run Now, or connect them to scheduled_jobs.
9	Report schedules have no screen (API only).	See the Reports Book.
10	Missed runs during downtime are not caught up by the scheduler.	After an outage, use Run now for the daily jobs of the missed day.

10. Document Sign-Off

Serial Number	Description	Owner	Status
1	No open items at the time of issue	iorta TechNXT	Closed

Sign Off Date	Signature	Approved By	Department & Project Role



Transforming Insurance with AI Innovation



10+
COUNTRIES
Global Presence



8
GLOBAL OFFICES
Strategically Located



11+
MARKETS SERVED
Across Continents



100%
COMMITMENT
To Excellence



MALAYSIA



INDIA



PHILIPPINES



THAILAND



VIETNAM



USA



ETHIOPIA



CAMBODIA



HONGKONG



SINGAPORE



Scan More Details